CHƯƠNG 4
THỰC HIỆN DỰ ÁN

Mục tiêu bài học

1. Hiểu được vai trò, vị trí và mức độ phức tạp của Nhóm Hoạt động Thực hiện trong vòng đời dự án CNTT. Thực hiện là giai đoạn tiêu hao nguồn lực nhiều nhất và sinh ra sản phẩm thực tế.
2. Nắm vững các hoạt động quản lý cốt lõi cần thực hiện để duy trì tiến độ, từ quản lý đội nhóm, đảm bảo chất lượng, đến quản lý tri thức. Nắm được vai trò của PM như một người lãnh đạo và loại bỏ rào cản.
3. Biết cách sử dụng Kế hoạch Quản lý Dự án (PMP) và Đường cơ sở làm kim chỉ nam để tạo ra các Sản phẩm Bàn giao (Deliverables). Tập trung vào việc giữ vững cam kết đã đặt ra.

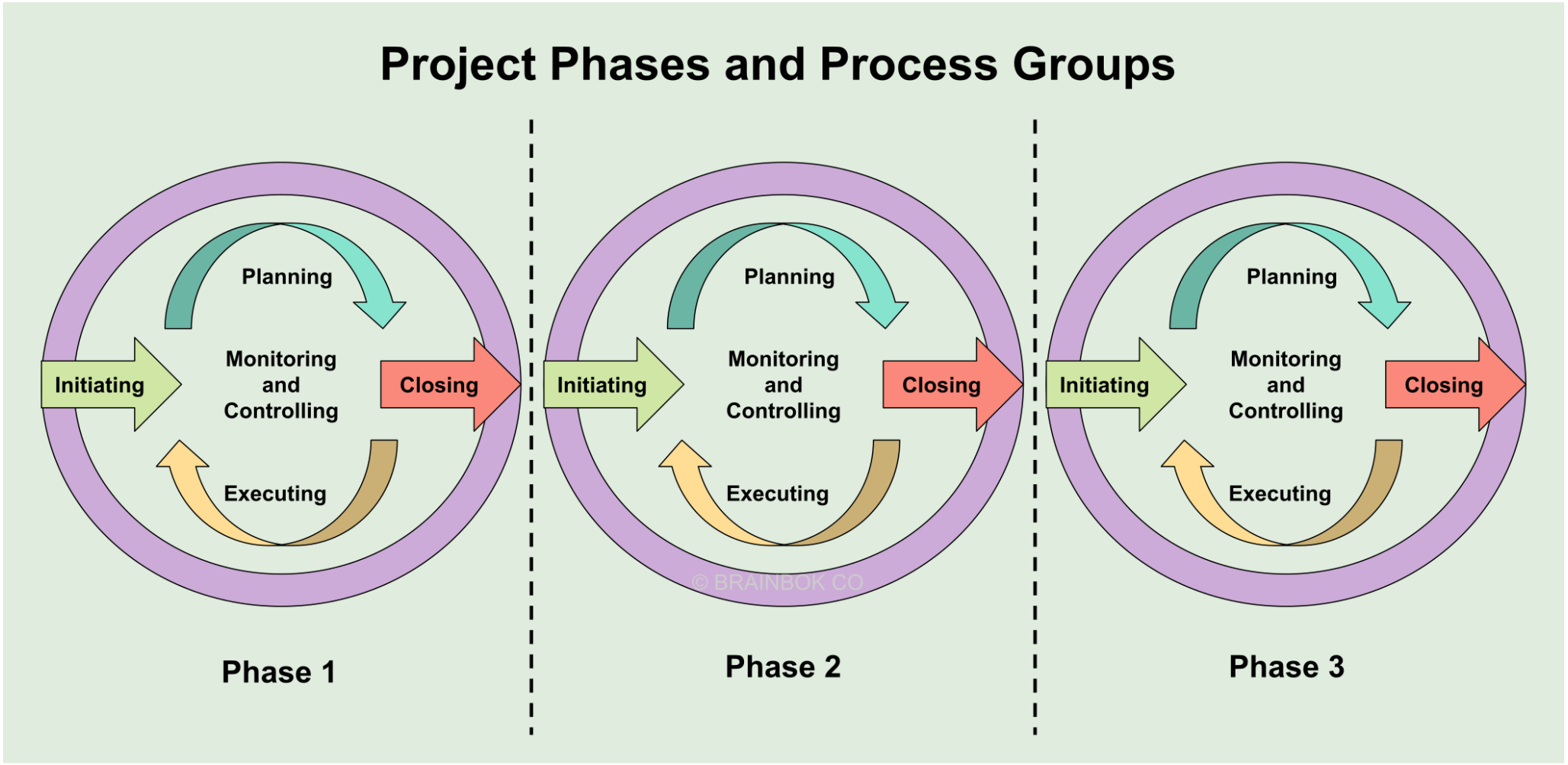
MIS
MANAGEMENT INFORMATION SYSTEMS
BANKING ACADEMY OF VIETNAM

Phần I: Tổng quan về Thực hiện dự án CNTT

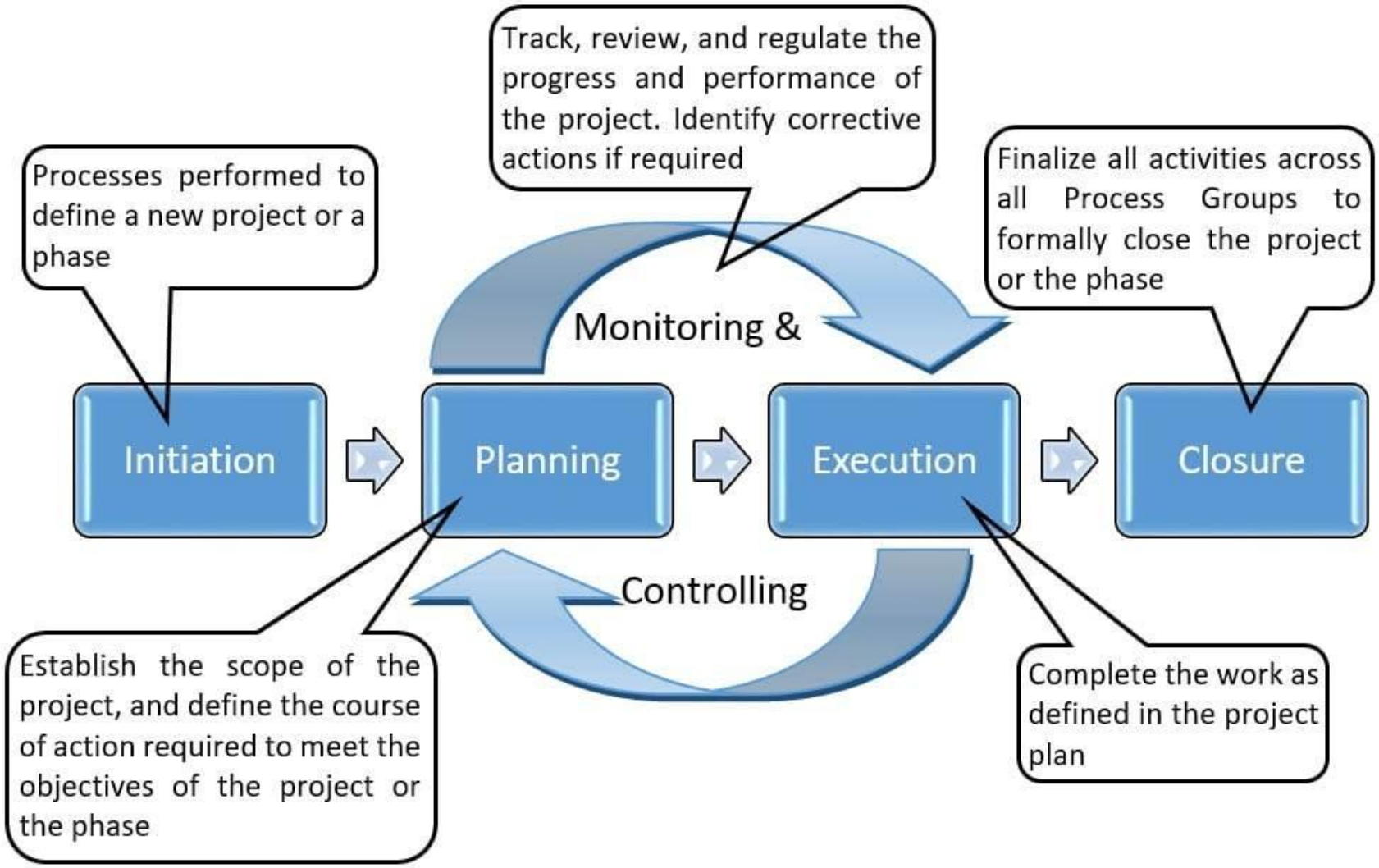
Tổng quan về thực hiện dự án

- Nhóm hoạt động thực hiện (Execution Group): bao gồm các quy trình thực hiện công việc đã được xác định trong Kế hoạch Quản lý Dự án (PMP) để đạt được các yêu cầu dự án.
- Đây là giai đoạn đòi hỏi nỗ lực lớn nhất (thường chiếm 50% - 70% tổng ngân sách và thời gian của dự án). Khác biệt lớn: chuyển từ việc Lập kế hoạch sang Thực thi kế hoạch.
- Nhiệm vụ chính: Thực hiện các gói công việc WBS (Code, Test, Deploy, Document, v.v.) để tạo ra các sản phẩm bàn giao kỹ thuật.
- Đầu vào: Kế hoạch Quản lý Dự án (PMP), bao gồm Đường cơ sở Phạm vi, Lịch trình, Chi phí, và các Kế hoạch Quản lý khác.

=> Không bắt đầu thực hiện nếu PMP chưa được phê duyệt chính thức.


Project Phases and Process Groups
Planning
Initiating
Monitoring and Controlling
Closing
Executing
Phase 1
Phase 2
Phase 3
© BRAINBOK CO

Vị trí của giai đoạn Thực hiện



Đầu vào cốt lõi của giai đoạn thực hiện

- Kế hoạch quản lý dự án (PMP)
- Yếu tố môi trường tổ chức (EEFS)
- Tài sản quy trình tổ chức (OPAS)
- Các yêu cầu thay đổi đã phê duyệt

Đầu vào cốt lõi của giai đoạn thực hiện

Kế hoạch Quản lý Dự án (PMP)	Là đầu vào quan trọng nhất, chứa **tất cả** các quyết định đã được phê duyệt ở giai đoạn Kế hoạch.	PMP là **sổ tay hướng dẫn** (Guidebook) cho mọi hoạt động trong giai đoạn này.

PMP bao gồm:

Đường cơ sở PMB (Performance Measurement Baseline)	**Phạm vi, Lịch trình, Chi phí** đã được khóa.	Bất kỳ sự sai lệch nào cũng phải được ghi nhận và xử lý qua **Kiểm soát Thay đổi**.
Kế hoạch Quản lý Chất lượng	Xác định **cách thức** kiểm tra Code Review, Test Coverage.	Hướng dẫn đội ngũ về tiêu chuẩn kỹ thuật phải đạt.
Kế hoạch Quản lý Nhân sự & Truyền thông	Ai làm gì (RACI), và cách thức báo cáo trạng thái.	Hỗ trợ điều phối và quản lý đội ngũ hàng ngày.

Yếu tố môi trường tổ chức (EEFs)

- Các yếu tố bên ngoài mà PM không thể kiểm soát nhưng phải thích nghi. PM phải làm việc trong khuôn khổ này.
- Trong CNTT, EEFs quan trọng:
  - Văn hóa Tổ chức: Mức độ ủng hộ đổi mới, khả năng chịu rủi ro.
  - Cơ sở hạ tầng Kỹ thuật: Các công cụ, hệ thống hiện có (CI/CD Pipelines, Source Code Repositories).
  - Tiêu chuẩn Chính phủ/Pháp lý: Quy định về Bảo mật Dữ liệu (GDPR, ISO 27001).

Tài sản quy trình tổ chức (OPAs)

- Các tài sản nội bộ mà PM **có thể sử dụng** để thực hiện dự án hiệu quả hơn.
- Các OPAs quan trọng:
  - Cơ sở Tri thức: Kho lưu trữ các Bài học Kinh nghiệm (Lessons Learned) từ các dự án trước.
  - Mẫu Biểu mẫu & Quy trình: Mẫu báo cáo trạng thái, quy trình kiểm thử đã được tiêu chuẩn hóa.
  - Cơ sở dữ liệu Rủi ro: Danh sách các rủi ro phổ biến trong các dự án CNTT nội bộ.

=> Giúp định hình chất lượng và quy trình làm việc.

Vai trò & Trách nhiệm chính trong giai đoạn thực hiện

❖ **Vai trò của Nhà Quản lý Dự án (PM)** PM là **Lãnh đạo và Điều phối viên** trung tâm của giai đoạn Thực hiện.

Lãnh đạo & Hướng dẫn	Đảm bảo đội ngũ hiểu và tuân thủ **PMP** và các **Đường cơ sở**.	PM là người bảo vệ PMP và Phạm vi dự án.
Quản lý Nguồn lực	Hỗ trợ **Thu nhận Nguồn lực** và giải quyết các vấn đề **quản lý đội ngũ** (xung đột, hiệu suất).	PM dành nhiều thời gian hơn cho **quản lý con người (HR)** và động lực làm việc.
Quản lý Tri thức	Đảm bảo **Bài học Kinh nghiệm (Lessons Learned)** được thu thập và áp dụng ngay trong quá trình thực hiện.	
Giải quyết Vấn đề	Nhận diện, phân tích và giải quyết các **Vấn đề (Issues)** và rào cản **hàng ngày** (ví dụ: lỗi tích hợp, thiếu quyền truy cập).	PM là người loại bỏ các **Blockers** để giữ cho đội ngũ CNTT có thể tiếp tục công việc.

Vai trò & Trách nhiệm chính trong giai đoạn thực hiện

❖ **Vai trò của Đội ngũ Dự án (Team Members)** — Đội ngũ kỹ thuật là những người **tạo ra giá trị** thực tế.

Trách nhiệm Chính	Thực hiện các **Gói Công việc** được giao (Develop, Code, Test, Deploy) theo **Yêu cầu Chất lượng** đã định.	Đội ngũ chịu trách nhiệm chính về **chất lượng kỹ thuật** và **ước tính thời gian** chi tiết.
Báo cáo Trạng thái	Cung cấp **Dữ liệu Hiệu suất Công việc (Work Performance Data)** trung thực và kịp thời cho PM (Tiến độ Code, Thời gian hoàn thành, v.v.).	Dữ liệu này là đầu vào quan trọng cho Nhóm **Giám sát & Kiểm soát**.
Quản lý Nội bộ	Tham gia tích cực vào các hoạt động phát triển nhóm và giải quyết xung đột nội bộ.	Tự chủ và trách nhiệm cá nhân cao (tính chất của CNTT).

❖ **Mối quan hệ PM và Đội ngũ** — **Phân biệt vai trò:** PM quản lý **QUY TRÌNH**, Đội ngũ quản lý **CÔNG VIỆC** kỹ thuật.

MIS
MANAGEMENT INFORMATION SYSTEMS
BANKING ACADEMY OF VIETNAM

Phần II: Thực hiện và quản lý công việc

Thực hiện & Quản lý công việc dự án

Mục đích: Thực hiện các hành động đã được xác định trong **Kế hoạch Quản lý Dự án (PMP)**, quản lý nhân sự, và tạo ra các **Sản phẩm Bàn giao (Deliverables)** đã lên kế hoạch.

Các hoạt động quản lý cốt lõi

- PM thực hiện các hoạt động điều phối:

✓ **Phân công Công việc (Work Authorization)**	Chính thức ủy quyền và gán các **Gói Công việc** (Work Packages) cho các thành viên/đội ngũ dự án.	Đảm bảo chỉ những công việc **đã được phê duyệt** mới được thực hiện.
✓ **Tích hợp Tri thức (Knowledge Integration)**	Áp dụng các **Bài học Kinh nghiệm (Lessons Learned)** và **Quy trình Tổ chức** (OPAs) vào công việc hàng ngày.	Trong CNTT: Sử dụng lại các đoạn code, thư viện, hoặc khuôn mẫu kiến trúc đã có sẵn.
✓ **Quản lý Thay đổi Kỹ thuật**	Thực hiện các **Yêu cầu Thay đổi Đã Phê duyệt** bằng cách điều chỉnh code, tài liệu, và lịch trình.	Cần được giao tiếp và **tích hợp ngay lập tức** vào quy trình CI/CD.
✓ **Quản lý Rủi ro Chủ động**	Thực hiện các hành động **Ứng phó Rủi ro** (Risk Response) khi các **Triggers** của rủi ro xảy ra.	Ví dụ: Nếu rủi ro 'Nhân sự chủ chốt nghỉ việc' xảy ra, PM kích hoạt **Kế hoạch Dự phòng** (Contingency Plan) đã định.

Kết quả của hoạt động thực hiện

Sản phẩm bàn giao (Deliverables)	Các kết quả hữu hình và vô hình được tạo ra (ví dụ: Module đã code, Báo cáo kiểm thử, Tài liệu người dùng).	Sau đó được chuyển sang quá trình **Kiểm soát Chất lượng** và **Xác nhận Phạm vi**.
Dữ liệu hiệu suất công việc (Work Performance Data - WPD)	Các số liệu thô về trạng thái dự án: **Thời gian thực tế đã chi tiêu**, **Tỷ lệ phần trăm đã hoàn thành**, **Số lượng lỗi**, **Ngày bắt đầu/kết thúc thực tế**.	**Đầu ra quan trọng nhất** — Dữ liệu này là cơ sở để **Giám sát & Kiểm soát**.
Yêu cầu thay đổi mới	Khi công việc thực tế bộc lộ vấn đề mới, PM tạo ra các Yêu cầu Thay đổi và gửi chúng đến **CCB**.	

Hệ thống PMIS và Công cụ CNTT trong Thực hiện

- Hệ thống Quản lý Thông tin Dự án (PMIS) là hệ thống tập hợp các công cụ và kỹ thuật được sử dụng để thu thập, tích hợp, và phân phối kết quả của các quy trình quản lý dự án.
- PMIS là nơi PM và Đội ngũ tương tác để lập kế hoạch, báo cáo, và theo dõi tiến độ.
- Vai trò PMIS trong Thực hiện: Cung cấp khả năng **phân công công việc**, **ghi nhận trạng thái**, và **tự động hóa** việc thu thập **Dữ liệu hiệu suất công việc (WPD)**.

=> Giảm thiểu công việc hành chính và tăng tính minh bạch.

Các Công cụ PMIS Chuyên dụng trong CNTT

Quản lý Yêu cầu (Requirements Management)	**Jira, Azure DevOps, Trello (với cấu trúc nâng cao).**	Dùng để ghi nhận, theo dõi, và quản lý vòng đời của **User Stories/Tasks/Bugs** từ khi tạo đến khi hoàn thành.
Kiểm soát Phiên bản (Version Control)	**Git (GitHub, GitLab, Bitbucket).**	Hệ thống bắt buộc để quản lý mã nguồn. Tích hợp chặt chẽ với quy trình **Code Review** và **CI/CD**.
Lập Kế hoạch & Lịch trình	**Microsoft Project, Asana, Monday.**	Dùng để theo dõi **Thời gian thực tế đã sử dụng** so với **Đường cơ sở Lịch trình** (Schedule Baseline).
Triển khai & Tự động hóa	**CI/CD Tools (Jenkins, GitHub Actions, GitLab CI).**	Tự động hóa việc kiểm tra chất lượng (Linter, Test Coverage) và triển khai. **Ghi nhận dữ liệu chất lượng.**

Lợi ích của PMIS trong Thực hiện

- Minh bạch (Transparency):
  - Mọi người đều thấy được trạng thái công việc và tiến độ của nhau, giảm nhu cầu họp báo cáo trạng thái.
  - Cần thiết cho nhóm phát triển phân tán và quản lý kỳ vọng của Khách hàng.
- Ra quyết định dựa trên Dữ liệu:
  - Dễ dàng trích xuất WPD (Số giờ thực tế, Tỷ lệ hoàn thành, Số Bug) để cung cấp cho quá trình Giám sát.
  - Hỗ trợ PM đưa ra Yêu cầu Thay đổi hoặc hành động điều chỉnh kịp thời.

Quản lý Yêu cầu Thay đổi và Tích hợp

- Tích hợp Thay đổi: Đảm bảo rằng các yêu cầu thay đổi (Change Requests - CRs) được *Phê duyệt* bởi *Ủy ban Kiểm soát Thay đổi (CCB)* được tích hợp một cách có kiểm soát và hệ thống vào dự án.
- PM không được phép tự ý thực hiện thay đổi; chỉ thực hiện CRs đã được *phê duyệt*.
- Đầu vào: **Yêu cầu Thay đổi Đã Phê duyệt** (Approved Change Requests - ACRs) và **Kế hoạch Quản lý Thay đổi**.
- Đầu ra: **Sản phẩm Bàn giao (Deliverables)** được sửa đổi, **Dữ liệu Hiệu suất Công việc** mới, **Cập nhật PMP** (Lịch trình, Phạm vi).

Thực thi quy trình kiểm soát thay đổi (Change Control Process)

✓ **Phân tích Tác động (Impact Analysis)**	Phân tích lại tác động của thay đổi lên **Ba Đường cơ sở** (Phạm vi, Lịch trình, Chi phí).	Mặc dù đã được phân tích trước khi phê duyệt, cần xác nhận lại tính khả thi và tài nguyên cần thiết khi thực hiện.
✓ **Truyền thông Thay đổi**	Thông báo chính thức về thay đổi cho **Đội ngũ Dự án** và các bên liên quan (Stakeholders).	Cần đảm bảo mọi người đều làm việc trên **Phiên bản Phạm vi (Scope Version)** mới nhất.
✓ **Cập nhật Tài liệu Kỹ thuật**	Điều chỉnh các tài liệu quan trọng như **Thông số Kỹ thuật (Specifications)**, **Thiết kế Kiến trúc (Architecture Design)** và **WBS** nếu cần thiết.	Phải đảm bảo tính đồng bộ giữa Code và Tài liệu (Source Code và Documentation).

Tích hợp thay đổi trong quy trình phát triển CNTT

Phân công Lại Công việc (Re-assignment)	Điều chỉnh và phân công lại **Gói Công việc** (Tasks) trong hệ thống PMIS (Jira).	Các công việc cũ có thể bị hủy, công việc mới được tạo ra với **Ước tính** (Estimation) mới.
Quản lý Phiên bản (Version Control)	Đảm bảo thay đổi được thực hiện trên **nhánh code** (Code Branch) phù hợp và được kiểm thử cẩn thận trước khi hợp nhất (Merge).	Tránh việc thay đổi làm hỏng các tính năng đã được code trước đó (Regression).
Kiểm soát Chất lượng Bắt buộc	Bất kỳ thay đổi nào cũng phải trải qua quy trình **Code Review** và **Kiểm thử Hồi quy (Regression Testing)** đầy đủ như quy định trong **Kế hoạch Chất lượng**.	Đảm bảo thay đổi không làm giảm chất lượng tổng thể của sản phẩm.

Quản lý Tri thức trong Dự án (Manage Project Knowledge)

- Mục đích: Đảm bảo rằng kiến thức được sử dụng để tối ưu hóa việc thực hiện dự án và được thu thập để chia sẻ trong tổ chức. Tránh "phát minh lại bánh xe" và lặp lại các lỗi lầm từ dự án trước.
- Phân loại Tri thức:
  - Tri thức Hữu hình (Explicit Knowledge): Dễ dàng mã hóa và lưu trữ (tài liệu, code base, báo cáo lỗi).
  - Tri thức Vô hình (Tacit Knowledge): Kinh nghiệm cá nhân, bí quyết, hiểu biết sâu sắc (khó tài liệu hóa).

=> Cần các chiến lược khác nhau để quản lý hai loại tri thức này.

Chiến lược Thu thập và Áp dụng Tri thức

- **Sử dụng Tri thức Hiện có (Apply OPA)**: Áp dụng các **Tài sản Quy trình Tổ chức (OPAs)** vào công việc.
  - Sử dụng các mẫu kiến trúc phần mềm đã được phê duyệt.
  - Áp dụng quy trình kiểm thử đã được chuẩn hóa.
- **Giao tiếp và Chia sẻ (Tacit Knowledge):** Xây dựng cơ chế để chuyển giao **Tri thức Vô hình**
  - Pair Programming (Lập trình cặp): Chia sẻ kinh nghiệm code tốt nhất.
  - Mentoring & Coaching: Huấn luyện trực tiếp.
  - Community of Practice: Nhóm chuyên môn chia sẻ giải pháp.
- **Thu thập Tri thức Mới (Lessons Learned):** Ghi lại những gì đã học được trong quá trình thực hiện
  - Khi một *vấn đề lớn* được giải quyết, ghi lại nguyên nhân, giải pháp, và điều gì có thể làm khác đi.
  - Tài liệu hóa các *Kỹ thuật Code* hoặc *Tích hợp* mới thành công.

Công cụ & Hệ thống Tri thức CNTT

- Kho tri thức (Knowledge Repositories): Nơi lưu trữ tập trung các bài học kinh nghiệm và tài liệu kỹ thuật. Sử dụng *Wiki (Confluence)*, *SharePoint*, hoặc *cơ sở dữ liệu Bài học Kinh nghiệm* chuyên dụng.
- Hệ thống Quản lý Tài liệu: Đảm bảo các phiên bản thiết kế và thông số kỹ thuật (Specifications) mới nhất luôn dễ dàng truy cập.

Quản lý Tri thức: Phân loại và Chiến lược trong CNTT

Tri thức Hữu hình (Explicit Knowledge) - "Cái gì"	Tri thức **dễ dàng tài liệu hóa, lưu trữ** và chia sẻ. Có thể được mã hóa, truyền đạt bằng ngôn ngữ chính thức.	○ **Mã nguồn (Code Base)** và Tài liệu Kỹ thuật. ○ **Báo cáo Lỗi** (Bug Reports) và Giải pháp. ○ **Kế hoạch Dự án** và Mẫu biểu.
Tri thức Vô hình (Tacit Knowledge) - "Làm thế nào"	Tri thức **dựa trên kinh nghiệm, trực giác, và bối cảnh** cá nhân. Khó khăn để chuyển giao bằng lời nói hoặc tài liệu.	○ **Bí quyết (Know-how)** xây dựng Kiến trúc phần mềm. ○ **Kỹ năng** gỡ lỗi (Debugging) phức tạp. ○ **Kinh nghiệm** ước tính hoặc quản lý rủi ro.

Tăng cường Tri thức Vô hình

- Lập trình Cặp (Pair Programming): Hai lập trình viên làm việc cùng nhau trên một nhiệm vụ/màn hình.

⇒Truyền đạt kỹ thuật code và tư duy giải quyết vấn đề.

- Huấn luyện (Coaching) và Cộng đồng Thực hành (CoP): Các lập trình viên có kinh nghiệm hướng dẫn người mới. Tạo ra các nhóm chuyên môn để chia sẻ các giải pháp cụ thể (Ví dụ: Nhóm Chuyên môn về Hiệu năng Database).

⇒Tăng tốc độ học hỏi và giảm rủi ro phụ thuộc vào một cá nhân.

Khai thác Tri thức Hữu hình

- Kho lưu trữ Tri thức (Knowledge Repositories): Cấu trúc hóa các tài liệu quan trọng (Wiki, Confluence, SharePoint) để dễ dàng tìm kiếm.

⇒**Tập trung** hóa thông tin thay vì để rải rác trong email hoặc ổ đĩa cá nhân.

- Thu thập Bài học Kinh nghiệm (Lessons Learned): Biến kinh nghiệm thực tế thành tài liệu chính thức sau khi giải quyết một **Vấn đề (Issue)** hoặc hoàn thành một **Sprint/Module**.

⇒Cung cấp **OPAs** cho các dự án tương lai.

Sử dụng bài học kinh nghiệm để cải tiến liên tục

- Mục đích: Đảm bảo các thành công được lặp lại và các lỗi được tránh khỏi trong các dự án sau này.

⇒ Cải thiện khả năng trưởng thành (Maturity) của tổ chức trong quản lý dự án CNTT.

- Vai trò của PM: PM là người chịu trách nhiệm chính thức hóa các kinh nghiệm (biến tri thức vô hình thành hữu hình).

⇒ Đảm bảo rằng các bài học kinh nghiệm (Lessons Learned) được ghi lại chi tiết, đầy đủ và dễ tìm kiếm.

Thu thập LL trong các họa động Thực hiện, triển khai

- Thu thập các bài học kinh nghiệm (LL) không chỉ được thực hiện ở cuối dự án; đây là một hoạt động liên tục.

- Thời điểm thu thập:
  - Sau khi một vấn đề lớn được giải quyết (Root Cause Analysis).
  - Sau mỗi Sprint/Giai đoạn quan trọng (Retrospective Meeting).
  - Sau khi hoàn thành sản phẩm bàn giao (Deliverable) phức tạp.

⇒Giảm thiểu lãng phí thời gian và nguồn lực do lặp lại các vấn đề đã xảy ra.

Bài học kinh nghiệm cốt lõi trong Dự án CNTT

- Độ chính xác Ước tính: So sánh Ước tính Ban đầu (Baseline) về thời gian và chi phí với Thực tế. Điều gì khiến ước tính sai lệch? (VD: Độ phức tạp của tích hợp hệ thống cũ bị đánh giá thấp).
- Hiệu suất Quy trình Kỹ thuật: Quy trình Code Review có đủ hiệu quả để bắt lỗi không? Môi trường CI/CD có ổn định không? Nếu quy trình CI/CD thường xuyên bị lỗi, cần tài liệu hóa cách khắc phục.
- Quản lý Rủi ro: Các Rủi ro nào đã xảy ra và Kế hoạch Ứng phó có hoạt động hiệu quả không? Nếu Kế hoạch Dự phòng không hiệu quả, cần sửa đổi để dùng cho dự án sau.

MIS
MANAGEMENT INFORMATION SYSTEMS
BANKING ACADEMY OF VIETNAM

Phần III: Quản lý đội ngũ

Thu nhận Nguồn lực

- Mục đích: Đảm bảo rằng các vai trò đã được xác định trong Kế hoạch Nguồn nhân lực được bổ sung bằng các cá nhân cụ thể.
- Đầu vào: Kế hoạch Nguồn nhân lực (HR Plan) và Ma trận RACI (Vai trò/Trách nhiệm).
- Đầu ra chính: Sự phân công nhân sự dự án (Project Staff Assignments) và Lịch Nguồn lực (Resource Calendar).

Thu nhận Nguồn lực trong Dự án CNTT

- **Nguồn nội bộ (Internal Resources)**
  - **Phân công trước (Pre-assigned):** Các thành viên đã được chỉ định trước khi dự án bắt đầu.
  - **Đàm phán/Điều phối:** PM đàm phán với các **Quản lý Chức năng (Functional Managers)** để có được lập trình viên, QA, hoặc chuyên gia DevOps.

⇒ Việc này đòi hỏi kỹ năng **đàm phán** và **ảnh hưởng** mạnh mẽ từ PM.

- **Nguồn bên ngoài (External Resources)**
  - **Thuê ngoài (Outsourcing):** Ký hợp đồng với các công ty bên ngoài để cung cấp dịch vụ code, kiểm thử, hoặc triển khai.
  - **Thuê nhân sự T&M (Time & Material):** Thuê các chuyên gia tư vấn/nhân sự có kỹ năng hiếm (ví dụ: chuyên gia Blockchain/AI).

⇒ Phải tuân thủ chặt chẽ **Kế hoạch Quản lý Mua sắm** (Procurement Plan).

Đưa đội ngũ vào công việc (On-boarding)

- Tổ chức buổi **Kick-off** chi tiết để giới thiệu **Phạm vi, Mục tiêu**, và đặc biệt là **PMP** (Đường cơ sở).

⇒ Giảm thời gian làm quen và hạn chế nhầm lẫn về mục tiêu.

- **Cấp Quyền truy cập Kỹ thuật:** Cấp quyền truy cập vào các công cụ thiết yếu: **PMIS** (Jira), **Version Control** (Git), **Môi trường Phát triển (IDE)**, và **Máy chủ/Cloud** (AWS/Azure).

- Cung cấp **Gói Thông tin Dự án** (Project Information Package) bao gồm **Kiến trúc Hệ thống**, **Coding Standards**, và **Lessons Learned** (OPAs).

⇒ Tận dụng **Tri thức Hữu hình** đã thu thập.

Đội ngũ CNTT Chuyên môn hóa: Các Vai trò Cốt lõi

- Đội ngũ dự án CNTT được xây dựng dựa trên các **chuyên môn sâu (Deep Specialization)**. PM cần hiểu rõ phạm vi kỹ năng để phân công và quản lý rủi ro. Các thành viên có thể là **T-Shaped** (có kỹ năng tổng quát rộng nhưng chuyên môn sâu ở một lĩnh vực).
- **Thách thức của PM:** Khi **phụ thuộc vào Chuyên gia Đơn lẻ** (Key-person Risk/Single Point of Failure) và **Rào cản Giao tiếp** giữa các nhóm chuyên môn khác nhau (Dev, QA, Infra).

Các vai trò kỹ thuật cốt lõi

Business Analyst (BA)	Cầu nối giữa **Business (Kinh doanh)** và **Technical (Kỹ thuật)**. Trách nhiệm chuyển đổi **Yêu cầu Kinh doanh** thành **User Stories** hoặc **Functional Specifications**.	Đảm bảo Code được viết **đúng** với nhu cầu của Khách hàng (Doing the right thing).
Lập trình viên (Developers)	Phát triển mã nguồn và xây dựng các thành phần của hệ thống.	
○ **Backend Dev**	Tập trung vào Logic nghiệp vụ, API, Database, và hiệu suất máy chủ.	Chịu trách nhiệm về **Hiệu năng** và **Bảo mật** (NFRs).
○ **Frontend Dev**	Tập trung vào Giao diện người dùng (UI/UX) và Trải nghiệm người dùng.	Chịu trách nhiệm về **Khả năng sử dụng (Usability)**.
QA/Tester (Quality Assurance)	Thiết kế và thực hiện các kịch bản kiểm thử (Unit, Integration, System, UAT).	Đảm bảo Code được viết **đúng cách** (Doing the thing right).
DevOps/Infra Engineer	Quản lý hạ tầng, môi trường, tự động hóa quy trình **CI/CD** (Continuous Integration/Continuous Delivery).	Đảm bảo sự **ổn định** của các môi trường (Dev, Test, Prod) và tốc độ triển khai.

Tối ưu hóa sự phân công

- **Ma trận Kỹ năng (Skill Matrix):** Công cụ trực quan để theo dõi và đối chiếu **nhu cầu kỹ năng** của dự án với **khả năng** của đội ngũ hiện có.

⇒ Giúp PM xác định chính xác các **khoảng trống kỹ năng** cần được đào tạo hoặc bổ sung.

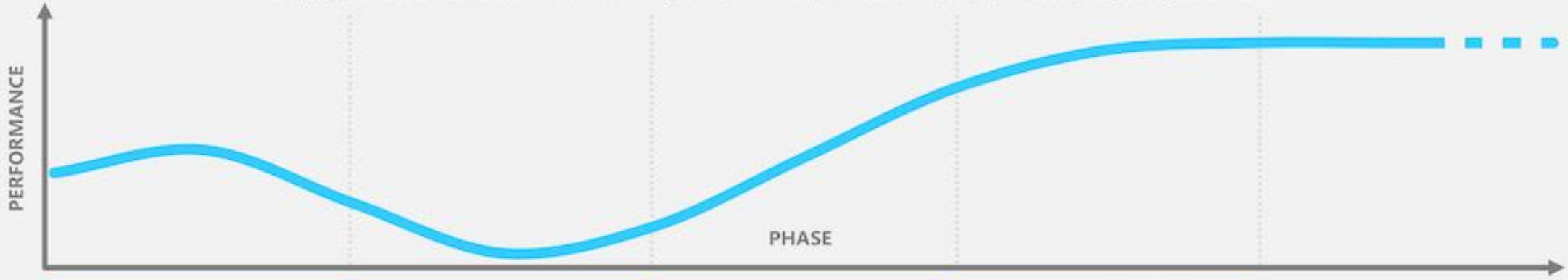
- **Quản lý Phụ thuộc:** PM phải đảm bảo các nhiệm vụ có sự phụ thuộc giữa các vai trò (ví dụ: Dev cần API của DevOps) được sắp xếp theo đúng trình tự (Sequencing).

Phases of Team Development

Forming, Storming, Norming, Performing, and Adjourning — based on group development model by Bruce Tuckman

All phases are necessary and inevitable for a team to grow, tackle problems, find solutions, plan work, and deliver results.

Copyright © 2008-2021 Scott M. Graffius. All rights reserved. For permission requests, contact scott@scottgraffius.com.



	FORMING	STORMING	NORMING	PERFORMING	ADJOURNING
CHARACTERISTICS	• Displaying eagerness • Socializing • Generally polite tone • Sticking to safe topics • Unclear about how one fits in • Some anxiety & questioning	• Some resistance • Lack of participation • Conflict based on differences of feelings & opinions • Competition • High emotions • Starting to move towards group norms	• Purpose & goals are well-understood • More confident • Improved commitment • Members are engaged and supportive • Relief, lowered anxiety • Developing cohesion	• High motivation, trust & empathy • Individuals defer to team needs • Effectively producing deliverables • Consistent performance • Demonstrations of interdependence & self-management	• (Also referred to as the Transitioning or Mourning phase) • Shift to process orientation • Sadness • Recognition of team & individual efforts • Disbanding
STRATEGIES	• Taking the 'lead' • Being highly visible • Facilitating introductions • Providing the 'big picture' • Establishing clear expectations • Communicating success criteria • Ensuring response times are quick	• Requesting & encouraging feedback • Identifying issues & facilitating their resolution • Normalizing matters • Building trust by honoring commitments	• Recognizing individual & team efforts • Providing learning opportunities & feedback • Monitoring the 'energy' of the team	• 'Guiding from the side' (minimal intervention) • Celebrating successes • Encouraging collective decision-making & problem-solving	• Recognizing change • Providing an opportunity for summative team evaluations ('lessons learned') • Providing an opportunity for individual acknowledgments • Celebrating the team's accomplishments (an 'after-party')

Graffius, Scott M. (2021). Phases of Team Development. Digital Object Identifier (DOI): 10.13140/RG.2.2.22040.42246.

v21010407

Forming (Hình thành)	Giai đoạn làm quen, tập trung vào nhiệm vụ, vai trò còn mơ hồ.	**Chỉ đạo (Directing):** Cung cấp định hướng rõ ràng, giới thiệu Phạm vi và Kế hoạch (PMP).
Storming (Xung đột)	Xung đột về quy trình, phong cách làm việc, và tranh giành quyền lực. **Giai đoạn nguy hiểm** nhất đối với sự tan rã của nhóm.	**Huấn luyện (Coaching):** Quản lý xung đột, khuyến khích sự đồng thuận về quy trình Code Review và Kiến trúc Kỹ thuật.
Norming (Ổn định)	Nhóm đạt được sự đồng thuận, thiết lập các quy tắc làm việc và quy trình đã được thống nhất.	**Hỗ trợ (Supporting):** Ủy quyền và hỗ trợ nhóm tự quản lý các quy trình nội bộ.
Performing (Hiệu quả)	Nhóm hoạt động hiệu quả, tự quản lý, và liên tục cải tiến. PM chỉ tập trung vào việc loại bỏ rào cản bên ngoài.	**Ủy quyền (Delegating):** Cung cấp cơ hội phát triển và bảo vệ nhóm khỏi sự gián đoạn bên ngoài.
Adjourning (Kết thúc)	Giai đoạn giải tán nhóm sau khi sản phẩm bàn giao được nghiệm thu.	**Ghi nhận & Chuyển đổi:** Đảm bảo **Lessons Learned** cuối cùng được ghi lại. Giải phóng và tái phân bổ nguồn lực (phối hợp với Quản lý Chức năng/PM khác).

Quản lý Nhóm: Đánh giá Hiệu suất và Quản lý Xung đột

- **Mục đích:** Đánh giá hiệu quả làm việc của đội ngũ **so với Kế hoạch** (PMP) và **Đường cơ sở** đã cam kết. Giúp xác định các **Khoảng trống Kỹ năng** cần được đào tạo hoặc can thiệp quản lý.
- **Công cụ Đánh giá (CNTT):**
  - **Velocity (Agile):** Tốc độ hoàn thành công việc của nhóm qua các Sprint.
  - **Tỷ lệ Hoàn thành Công việc:** So sánh nhiệm vụ được giao với nhiệm vụ đã hoàn thành.
  - **Số lượng Bug (Defects):** Số lỗi được tìm thấy sau khi code hoàn thành (đầu ra của QA).

⇒Số liệu trực tiếp chuyển thành **Thông tin Hiệu suất Công việc** (Work Performance Information).

Quản lý Xung đột (Conflict Management)

- **Quản lý Xung đột (Conflict Management):** Xung đột là không thể tránh khỏi (đặc biệt trong giai đoạn **Storming**), PM phải giải quyết chúng kịp thời.
- **Các Nguồn xung đột Phổ biến (CNTT):**
  - **Ưu tiên Lịch trình:** Tranh cãi về nhiệm vụ nào nên được ưu tiên hơn.
  - **Kiến trúc/Kỹ thuật:** Bất đồng về giải pháp kỹ thuật tốt nhất (ví dụ: dùng công cụ nào, cách thiết kế cơ sở dữ liệu).
  - **Nguồn lực Cá nhân:** Cảm giác phân bổ không công bằng hoặc quá tải.

Xung đột kỹ thuật (Technical Conflict) thường là **xung đột mang tính xây dựng** (Constructive Conflict).

Phương pháp Giải quyết Xung đột (5 Phương thức)

PM cần linh hoạt áp dụng các kỹ thuật khác nhau:

Tránh né (Withdraw/Avoid)	Hoàn toàn né tránh vấn đề.	**Chỉ dùng** khi vấn đề không quan trọng. **Thường không hiệu quả.**
San bằng/Hòa giải (Smooth/Accommodate)	Nhấn mạnh sự đồng thuận thay vì giải quyết vấn đề gốc rễ.	**Chỉ dùng** khi cần duy trì mối quan hệ và vấn đề nhỏ.
Thỏa hiệp (Compromise/Reconcile)	Cả hai bên đều từ bỏ một phần yêu cầu để đạt được giải pháp tạm thời.	**Giải pháp Tạm thời**, đôi khi dẫn đến giải pháp kém tối ưu.
Cưỡng chế (Force/Direct)	PM sử dụng quyền lực để áp đặt giải pháp.	**Nhanh chóng**, nhưng thường làm tổn hại đến mối quan hệ. **Chỉ dùng** trong trường hợp khẩn cấp.
Giải quyết Vấn đề (Problem Solve/Confront)	**Phương pháp Tốt nhất:** Tập trung vào **Giải pháp** và **Mục tiêu Dự án**, không phải cá nhân.	**Tập trung vào Nguyên nhân Gốc rễ** (Root Cause), đạt được sự đồng thuận lâu dài.

Động lực làm việc của Đội ngũ

PM cần hiểu những gì thực sự thúc đẩy đội ngũ kỹ thuật. Developer thường được thúc đẩy bởi yếu tố nội tại hơn là chỉ tiền bạc.

- **Tự chủ (Autonomy):** Cho phép đội ngũ **tự quyết định** về cách thức thực hiện công việc, kỹ thuật lập trình, hoặc lựa chọn công cụ trong khuôn khổ Kiến trúc đã định.

⇒ Giảm sự can thiệp vi mô (Micromanagement) của PM vào các quyết định kỹ thuật.

- **Thành thạo (Mastery):** Cung cấp các **thử thách kỹ thuật** phức tạp và cơ hội học hỏi, phát triển kỹ năng mới, như làm việc với các công nghệ tiên tiến (AI, Cloud, Big Data).

⇒ PM cần lập kế hoạch **Đào tạo** và cho phép **Thời gian Đổi mới** (Innovation Time).

- **Mục đích (Purpose):** Đảm bảo đội ngũ hiểu rõ **giá trị** mà sản phẩm của họ mang lại cho khách hàng (Why we code?).

⇒ Kết nối công việc Code hàng ngày với **Mục tiêu Kinh doanh** lớn hơn.

Khen thưởng & Ghi nhận

- Ghi nhận là một phần cốt lõi của **Quản lý Nhóm** (Manage Team).
  - **Ghi nhận Chính thức (Formal Recognition)**
    - **Phần thưởng Tiền tệ:** Tiền thưởng hoàn thành dự án dựa trên **Đường cơ sở Chi phí** (Cost Baseline).
    - **Cơ hội Nghề nghiệp:** Thăng chức, trao quyền lãnh đạo kỹ thuật (Ví dụ: Tech Lead).

  ⇒Liên kết trực tiếp với **Đánh giá Hiệu suất** (Slide 16) và **KPI** đã định.
  - **Ghi nhận Phi chính thức (Informal Recognition)**
    - **Phản hồi Cá nhân Kịp thời:** Khen ngợi công khai về một đoạn code sạch, hay việc giải quyết một lỗi phức tạp.
    - **Linh hoạt:** Cấp thêm thời gian làm việc từ xa, hoặc quyền chọn mua **công cụ làm việc** tốt hơn.

  ⇒**Rất hiệu quả** vì thể hiện sự tin tưởng và tôn trọng đối với chuyên môn của kỹ sư.
- Khen thưởng phải phù hợp với văn hóa kỹ thuật và mang tính cá nhân hóa.
- Phản hồi không nên chỉ xảy ra hàng quý mà phải là một phần của **Daily Stand-up** và **Sprint Review**. Phản hồi kịp thời giúp PM củng cố hành vi tốt (viết code sạch, hoàn thành đúng hẹn) và điều chỉnh kịp thời các vấn đề về chất lượng.

Tổng kết